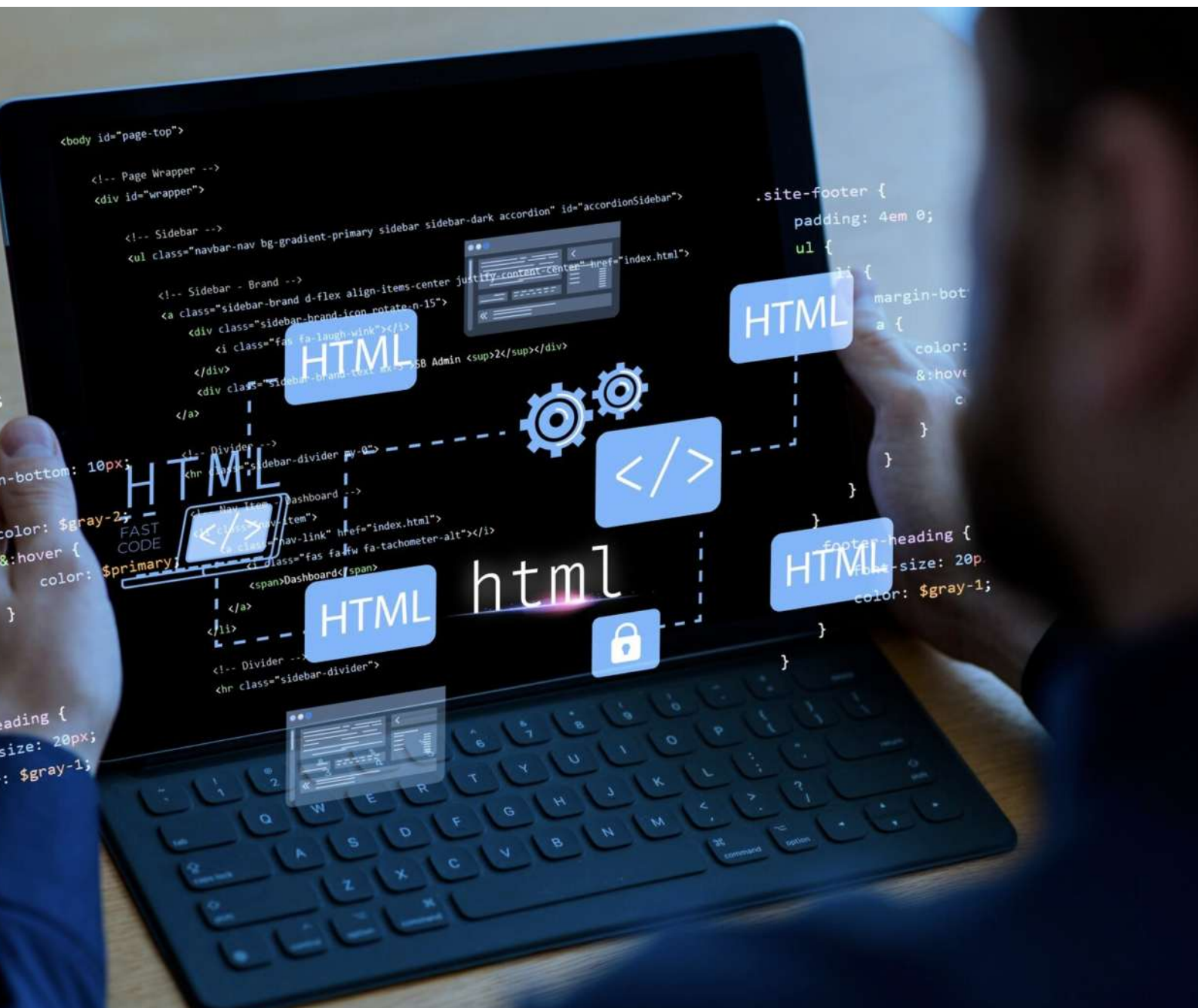


CURSO TEST AUTOMATION ENGINEER FORMACIÓN INTEGRAL



Objetivo General del Curso

ANALIZAR LAS HERRAMIENTAS DE DISEÑO DE TEST AUTOMATION ENGINEER, DE ACUERDO A LAS APLICACIONES WEB, MÓVILES Y APIS.

Objetivo específico del Módulo

RECONOCER CARACTERISTICAS DEL CONTROL DE VERSIONES Y ENTORNOS DE DESARROLLO, DE ACUERDO A LAS APLICACIONES WEB, MÓVILES Y APIS.

Contenidos

Objetivo General del Curso	2
Objetivo específico del Módulo.....	2
Módulo 2: CONTROL DE VERSIONES Y ENTORNOS DE DESARROLLO	4
Capítulo 1: GIT: FLUJO DE TRABAJO PROFESIONAL (BRANCHES, PULL REQUESTS, CONFLICTOS).....	6
Branches (Ramas)	6
Pull Requests (Solicitudes de extracción).....	7
Conflictos.....	8
Buenas prácticas para evitar conflictos:.....	10
Capítulo 2: GITHUB: USO COLABORATIVO DE REPOSITORIOS.....	11
1. Repositorios compartidos	11
2. Forks y Pull Requests	12
3. Trabajo con ramas y colaboración.....	12
4. Revisión de código (Code Review)	13
5. Automatización y pruebas (CI/CD).....	13
6. Organización y gestión de tareas	14
7. Seguridad y control de acceso	14
Capítulo 3: TERMINAL Y AUTOMATIZACIÓN DE TAREAS BÁSICAS.....	15
¿Qué es la terminal?	15
Comandos básicos de la terminal.....	15
Automatización de tareas	16
Automatización con herramientas externas	18
Buenas prácticas	19
Capítulo 4: INSTALACIÓN DE HERRAMIENTAS: NODE.JS, VS CODE, GESTORES DE PAQUETES.....	20
1. Node.js.....	20
2. Visual Studio Code (VS Code)	21
3. Gestores de paquetes.....	22
Buenas prácticas	24

Módulo 2: CONTROL DE VERSIONES Y ENTORNOS DE DESARROLLO



En el ámbito del desarrollo de software, el control de versiones y los entornos de desarrollo son pilares fundamentales para garantizar un trabajo colaborativo, ordenado y eficiente.

El control de versiones permite registrar, rastrear y gestionar los cambios realizados en el código fuente a lo largo del tiempo, facilitando la colaboración entre múltiples desarrolladores y la recuperación de versiones anteriores en caso de errores.

Por su parte, los entornos de desarrollo proporcionan las herramientas y configuraciones necesarias para programar, compilar, probar y desplegar aplicaciones de manera coherente y productiva.

Reconocer las características de estos componentes resulta esencial para adaptarse a los flujos de trabajo modernos y contribuir de manera efectiva a proyectos tecnológicos de cualquier escala.



Capítulo 1: GIT: FLUJO DE TRABAJO PROFESIONAL (BRANCHES, PULL REQUESTS, CONFLICTOS)

Git es un sistema de control de versiones distribuido ampliamente utilizado en el desarrollo de software. Permite a múltiples desarrolladores trabajar simultáneamente en un mismo proyecto, manteniendo un historial detallado de los cambios realizados. En un entorno profesional, se emplean ciertos flujos de trabajo que aseguran organización, eficiencia y control sobre el código.

A continuación, se detallan los elementos del flujo de trabajo profesional en Git:

Branches (Ramas)

Las branches o ramas son líneas de desarrollo independientes dentro de un repositorio. La práctica profesional más común es mantener una rama principal (como main o master) que representa la versión estable del proyecto, y trabajar en nuevas características, correcciones de errores o pruebas desde ramas secundarias. Algunas ramas comunes son:

- feature/nueva-funcionalidad: para nuevas características.
- bugfix/correccion-error: para resolver errores.
- hotfix/parche-urgente: para correcciones críticas en producción.
- develop: en algunos flujos como Git Flow, para integrar todas las ramas antes de pasar a producción.

Ventajas:

- Aislar cambios sin afectar el código estable.
- Facilitar pruebas y revisiones.
- Promover la colaboración paralela.

Pull Requests (Solicitudes de extracción)

Una pull request (PR) es una solicitud para fusionar una rama con otra, generalmente desde una rama de desarrollo hacia main. Este proceso permite:

- Revisar el código antes de integrarlo.
- Discutir y proponer mejoras.
- Ejecutar pruebas automáticas o validaciones.

Las PR fomentan la revisión entre pares (code review), una buena práctica profesional que mejora la calidad del software y promueve el aprendizaje en equipo.

Conflictos

En el contexto del desarrollo colaborativo, es común que varios miembros del equipo trabajen simultáneamente sobre el mismo proyecto. Git, como sistema de control de versiones distribuido, permite fusionar (merge) distintos aportes, pero en algunos casos no puede resolver automáticamente las diferencias entre versiones de archivos. A estos casos se les conoce como conflictos.

¿Cuándo ocurren los conflictos?

Un conflicto sucede principalmente cuando dos ramas modifican la misma línea de código o el mismo bloque dentro de un archivo, y Git no puede determinar cuál de las versiones es la correcta. Por ejemplo, si dos desarrolladores editan el mismo párrafo de un documento o la misma función de un archivo en ramas diferentes y luego intentan fusionarlas, Git generará un conflicto que debe resolverse manualmente.

¿Cómo identificar un conflicto?

Al intentar hacer un merge o un pull y se detecta un conflicto, Git detiene el proceso y muestra un mensaje indicando los archivos afectados. Al abrir el archivo en conflicto, Git delimita claramente las secciones problemáticas usando marcas especiales:

```
markdown

<<<<<< HEAD
Versión actual (rama activa)
=====
Versión entrante (rama que se intenta fusionar)
>>>>>> rama-secundaria
```

Estas marcas indican al desarrollador que debe elegir, combinar o reescribir el contenido para resolver el conflicto.

¿Cómo resolver un conflicto paso a paso?

- Revisar los archivos en conflicto: Git los marcará automáticamente en el editor o en la línea de comandos. Se pueden listar con `git status`.
- Editar manualmente el archivo: El desarrollador debe leer ambas versiones del contenido (entre <<<<<< y =====, y entre ===== y >>>>>>) y decidir cuál conservar, combinar o modificar.
- Eliminar las marcas de conflicto: Una vez elegida la versión correcta, se deben borrar todas las marcas (<<<<<<, =====, >>>>>>) para que el archivo quede limpio.
- Marcar el conflicto como resuelto: Se hace con el comando `git add <archivo>`.
- Realizar un commit: Se guarda el resultado final con `git commit`. Este commit registrará la resolución del conflicto.
- Continuar con el flujo: Si el conflicto surgió durante una fusión (merge) o un rebase, se puede continuar el proceso una vez resueltos todos los conflictos.

Herramientas para facilitar la resolución

Existen editores y herramientas gráficas que simplifican la visualización y resolución de conflictos, como:

- Visual Studio Code (resaltado y botones para aceptar cambios).
- GitKraken.
- SourceTree.
- Meld o Beyond Compare (para comparar archivos).

Estas herramientas permiten ver claramente las diferencias y elegir entre las versiones o combinarlas.

Buenas prácticas para evitar conflictos:

Aunque los conflictos no se pueden evitar completamente, sí se pueden reducir o mitigar mediante buenas prácticas:

- **Actualizar las ramas con frecuencia:** Ejecutar git pull con regularidad ayuda a mantener el código sincronizado con la rama principal, reduciendo diferencias acumuladas.
- **Hacer commits pequeños y enfocados:** Dividir los cambios en partes manejables facilita el rastreo de errores y disminuye el riesgo de sobreposición de código.
- **Trabajar con ramas temáticas:** Usar ramas específicas por funcionalidad, error o tarea ayuda a mantener el código organizado.
- **Evitar trabajar demasiado tiempo en una misma rama sin integrarse al flujo principal:** Las ramas de larga duración suelen acumular cambios que luego generan conflictos difíciles de manejar.
- **Comunicar los cambios relevantes:** Informar al equipo cuando se modifican archivos sensibles o compartidos permite coordinar esfuerzos y evitar ediciones simultáneas en la misma sección de código.

Capítulo 2: GITHUB: USO COLABORATIVO DE REPOSITORIOS

GitHub se ha convertido en una de las plataformas más utilizadas a nivel mundial para el desarrollo colaborativo de software. Esta herramienta basada en la nube complementa el uso de Git, proporcionando una interfaz visual, herramientas de colaboración, gestión de proyectos y control de acceso sobre los repositorios.

Entender cómo utilizar GitHub de forma colaborativa es esencial para equipos de trabajo que desean mantener una comunicación fluida, un control ordenado del código fuente y un flujo de integración continua que mejore la productividad y calidad del desarrollo.

GitHub permite que múltiples desarrolladores trabajen sobre un mismo proyecto de manera estructurada y eficiente, facilitando la integración de cambios, la revisión de código y la gestión de tareas. A continuación, se explican los aspectos de su uso colaborativo:



1. Repositorios compartidos

En GitHub, un repositorio (repo) es el contenedor del proyecto: incluye código, documentación, archivos de configuración y todo lo necesario para el desarrollo. Cuando se trabaja en equipo, los colaboradores pueden tener distintos niveles de acceso:

- **Propietario:** controla todo el repositorio (configuraciones, accesos, etc.).
- **Colaboradores:** tienen permisos para clonar, crear ramas, enviar pull requests y, en algunos casos, fusionar cambios.
- **Contribuyentes externos:** pueden proponer cambios a través de forks y pull requests si no tienen acceso directo.

Los roles se gestionan desde la pestaña Settings > Collaborators en el repositorio.

2. Forks y Pull Requests

Cuando un desarrollador externo desea contribuir a un proyecto sin permisos de escritura, puede hacer un fork. Este proceso crea una copia del repositorio en su cuenta personal, desde donde puede trabajar libremente.

- Flujo típico con fork:
- El usuario hace un fork del repositorio original.
- Clona el fork localmente y crea una nueva rama.
- Realiza los cambios y hace commit.
- Envía los cambios al fork con git push.
- Abre un pull request hacia el repositorio original para que los cambios sean revisados e integrados.

Este flujo promueve una colaboración abierta, especialmente en proyectos de código abierto.

3. Trabajo con ramas y colaboración

GitHub promueve el uso de ramas para organizar el desarrollo colaborativo. Algunas ramas habituales son:

- main o master: rama estable del proyecto.
- develop: integración de nuevas características.
- feature/, bugfix/, hotfix/: ramas específicas por tipo de tarea.

Cada colaborador puede crear una rama desde la interfaz o desde su entorno local, trabajar en ella, y luego subirla a GitHub. Posteriormente, se crea un pull request para fusionar esa rama con la rama principal del proyecto.

4. Revisión de código (Code Review)

Antes de fusionar los cambios propuestos en un pull request, GitHub permite que otros miembros del equipo:

- Revisen línea por línea el código nuevo.
- Dejen comentarios.
- Sugieran mejoras o correcciones.
- Aprueben o rechacen la solicitud.

Esto mejora la calidad del código, fomenta el aprendizaje colectivo y evita errores en producción.

5. Automatización y pruebas (CI/CD)

GitHub se integra con múltiples herramientas de integración continua como GitHub Actions, Travis CI, CircleCI, entre otras, que permiten:

- Ejecutar pruebas automáticamente al recibir un pull request.
- Verificar que el nuevo código no rompa funcionalidades existentes.
- Realizar despliegues automáticos si las pruebas son exitosas.
- Estas prácticas mejoran la confiabilidad del software y reducen errores humanos.

6. Organización y gestión de tareas

GitHub ofrece herramientas complementarias para organizar el trabajo:

- **Issues:** para reportar errores, proponer mejoras o registrar tareas.
- **Projects:** tableros tipo kanban para seguir el progreso del desarrollo.
- **Milestones:** permiten agrupar issues y pull requests según entregables o versiones del proyecto.

Estas funcionalidades permiten que incluso grandes equipos puedan coordinarse sin perder el control sobre el avance.

7. Seguridad y control de acceso

GitHub permite establecer:

- Branch protection rules: reglas para proteger ramas clave (como main) y evitar que se hagan cambios sin revisiones o pruebas exitosas.
- Revisión obligatoria de código antes del merge.
- Firmas de commits, escaneo de vulnerabilidades, y otras prácticas de seguridad.

El uso colaborativo de GitHub transforma el trabajo en equipo, facilitando la integración, revisión, automatización y despliegue de proyectos de software. Dominar estas herramientas y flujos de trabajo no solo mejora la eficiencia técnica, sino que también fortalece la comunicación y cohesión dentro del equipo de desarrollo.

Capítulo 3: TERMINAL Y AUTOMATIZACIÓN DE TAREAS BÁSICAS

La terminal es una herramienta esencial para los desarrolladores, ya que permite interactuar directamente con el sistema operativo a través de comandos. A través de ella se pueden ejecutar tareas de forma rápida, eficiente y automatizada.

Dominar el uso de la terminal y la automatización de tareas básicas no solo agiliza el flujo de trabajo, sino que también permite controlar versiones, compilar proyectos, ejecutar scripts, gestionar archivos y más, todo sin depender de interfaces gráficas. Esta habilidad es especialmente relevante en entornos de desarrollo profesional y en la administración de servidores o proyectos complejos.

¿Qué es la terminal?

La terminal, también conocida como consola o línea de comandos, es una interfaz textual que permite a los usuarios interactuar directamente con el sistema operativo. A diferencia de una interfaz gráfica, en la terminal las instrucciones se escriben mediante comandos, lo que otorga mayor control y flexibilidad sobre el sistema.

Existen distintos tipos de terminales, como:

- Bash (Bourne Again SHell): muy usada en sistemas Linux y macOS.
- CMD o PowerShell: en sistemas Windows.
- Terminal integrada en editores como Visual Studio Code.

Comandos básicos de la terminal

Algunos comandos esenciales para la gestión de archivos y directorios son:

Comando	Descripción
pwd	Muestra la ruta del directorio actual
ls / dir	Lista los archivos y carpetas en el directorio actual
cd <directorio>	Cambia de directorio
mkdir <nombre>	Crea una nueva carpeta
rm <archivo>	Elimina un archivo
rm -r <carpeta>	Elimina una carpeta y su contenido
touch <archivo>	Crea un archivo vacío (Linux/macOS)
echo <texto>	Muestra texto o lo guarda en un archivo
cat <archivo>	Muestra el contenido de un archivo
cp <origen> <destino>	Copia archivos o carpetas
mv <origen> <destino>	Mueve o renombra archivos

Automatización de tareas

La automatización permite ejecutar secuencias de comandos sin intervención manual. Esto es útil para compilar código, iniciar servidores, realizar pruebas, hacer backups o desplegar aplicaciones.

Existen dos formas comunes de automatizar tareas básicas:

1. Uso de scripts

Los scripts son archivos que contienen una secuencia de comandos. Pueden ser:

- Shell scripts (.sh) para sistemas Unix.
- Batch scripts (.bat) para Windows.
- Archivos .ps1 para PowerShell.

Ejemplo de shell script (Linux/macOS):

```
bash

#!/bin/bash
echo "Iniciando el proyecto..."
cd /ruta/del/proyecto
git pull
npm install
npm run start
```

Para ejecutarlo:

```
bash

chmod +x mi_script.sh    # Da permisos de ejecución
./mi_script.sh
```


2. Alias y funciones personalizadas

Puede definir alias en su terminal para abreviar comandos largos o repetitivos:

```
bash

alias iniciarproyecto="cd ~/mi-proyecto && npm run dev"
```

Esto se suele agregar al archivo `.bashrc`, `.zshrc` o `.bash_profile`, dependiendo del sistema.

También se pueden crear funciones más complejas:

```
bash

deploy() {
  git add .
  git commit -m "$1"
  git push
}
```

Luego puede usar:

```
bash

deploy "mensaje del commit"
```

Automatización con herramientas externas

Algunas herramientas ayudan a automatizar tareas sin necesidad de crear scripts desde cero:



Makefile:

Muy usado en proyectos C/C++ pero adaptable a cualquier flujo.



npm scripts:

Permite automatizar tareas dentro de proyectos Node.js.



Crontab (Linux): permite programar la ejecución automática de tareas en determinados horarios.



Task Scheduler

App

Task Scheduler (Windows): similar a crontab, pero con interfaz gráfica.

Buenas prácticas

- Comente sus scripts para facilitar su lectura y mantenimiento.
- Use rutas relativas cuando sea posible, para mejorar la portabilidad.
- Maneje errores usando condicionales y validaciones (if, exit, etc.).
- Evite automatizar acciones destructivas sin confirmación (rm -rf por ejemplo).
- Revise los permisos de ejecución y las rutas utilizadas.

El dominio de la terminal y la automatización básica no solo ahorra tiempo, sino que transforma el flujo de trabajo en uno más ágil, reproducible y controlado. Además, es una competencia transversal que potencia la productividad en múltiples áreas del desarrollo de software, desde la programación hasta la administración de sistemas.

Capítulo 4: INSTALACIÓN DE HERRAMIENTAS: NODE.JS, VS CODE, GESTORES DE PAQUETES

Antes de comenzar cualquier proyecto de desarrollo moderno, es fundamental contar con un entorno de trabajo configurado correctamente. Esto implica la instalación de herramientas esenciales como Node.js, Visual Studio Code (VS Code) y gestores de paquetes, que permiten automatizar la gestión de dependencias y facilitar la escritura y ejecución del código. Tener un entorno bien instalado y comprendido es el primer paso para desarrollar de forma profesional, eficiente y con las mejores prácticas de la industria.

1. Node.js



¿Qué es?

Node.js es un entorno de ejecución de JavaScript del lado del servidor. Permite desarrollar aplicaciones web escalables, scripts de automatización, herramientas de desarrollo y mucho más, utilizando JavaScript fuera del navegador.

¿Por qué es importante?

- Permite ejecutar JS sin navegador.
- Es la base de herramientas modernas de desarrollo frontend y backend (como React, Vue, Angular, Express, etc.).
- Incluye npm, el gestor de paquetes más usado en la actualidad.

Instalación de Node.js:

- Ir al sitio oficial: <https://nodejs.org>
- Descargar la versión LTS (Long Term Support), recomendada para la mayoría de usuarios.
- Ejecutar el instalador y seguir los pasos.
- Verificar la instalación desde la terminal:

```
bash

node -v      # Verifica la versión de Node.js
npm -v       # Verifica la versión del gestor de paquetes npm
```

💡 En macOS y Linux también puedes usar gestores como Homebrew (`brew install node`) o nvm para gestionar múltiples versiones.

2. Visual Studio Code (VS Code)



Visual Studio Code

¿Qué es?

Visual Studio Code es un editor de código fuente desarrollado por Microsoft. Es ligero, rápido y extensible, ideal para desarrollar en múltiples lenguajes.

Características destacadas:

- Autocompletado inteligente y depuración.
- Integración con Git.
- Terminal integrada.
- Gran ecosistema de extensiones (linters, themes, snippets, etc.).
- Compatible con Windows, macOS y Linux.

Instalación:

- Ir a <https://code.visualstudio.com/>
- Descargar la versión adecuada para tu sistema operativo.
- Ejecutar el instalador.

Extensiones recomendadas para desarrollo web:

- ESLint (análisis de código).
- Prettier (formateador de código).
- Live Server (actualización en tiempo real).
- GitLens (historial detallado de Git).
- IntelliCode (autocompletado inteligente basado en IA).

3. Gestores de paquetes

¿Qué son?

Los gestores de paquetes permiten instalar, actualizar y desinstalar dependencias necesarias para los proyectos. Automatizan la gestión de bibliotecas, frameworks y utilidades, asegurando que el entorno sea reproducible y fácil de mantener.

A. npm (Node Package Manager)

- Se instala automáticamente con Node.js.
- Usa el archivo package.json para describir el proyecto y sus dependencias.

```
bash
```

```
npm init           # Inicializa un proyecto (crea package.json)
npm install <paquete>  # Instala un paquete
npm install         # Instala todas las dependencias listadas
npm uninstall <paquete> # Elimina un paquete
```

B. Yarn (alternativa a npm)

- Más rápido en algunas operaciones.
- Administra mejor la consistencia de dependencias entre proyectos.

Instalación:

```
bash  
  
npm install -g yarn
```

Comandos equivalentes:

```
bash  
  
yarn init  
yarn add <paquete>  
yarn remove <paquete>
```

C. nvm (Node Version Manager)

Permite instalar y cambiar entre distintas versiones de Node.js.

Instalación en Unix/macOS:

```
bash  
  
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.3/install.sh | bash
```

Comandos básicos:

```
bash

nvm install 18      # Instala Node.js versión 18
nvm use 18          # Usa la versión 18
nvm list            # Lista versiones instaladas
```

Buenas prácticas

- Use la versión LTS de Node.js para mayor estabilidad.
- Mantenga sus paquetes actualizados (npm outdated).
- Revise y controle las versiones en package.json.
- No suba la carpeta node_modules a Git; usa .gitignore.
- Aproveche las extensiones de VS Code para mejorar la productividad.

La instalación y uso correcto de Node.js, Visual Studio Code y los gestores de paquetes constituyen la base del entorno de desarrollo moderno.

Estas herramientas, además de facilitar la codificación, el control de dependencias y la automatización de tareas, permiten establecer flujos de trabajo profesionales, reproducibles y escalables, fundamentales para el desarrollo de software actual.